

to. Si no podemos abortarla, simplemente la terminamos y luego iniciamos la lectura correcta. En este ejemplo POPI requiere 15 microinstrucciones en lugar de las 13 de la figura 4-16. Sin embargo, si el uso del traslape reduce el tiempo básico de microinstrucción en 15%, incluso POPI será más rápida que antes.

La situación se volvería en extremo compleja si se utilizaran saltos condicionales de alternativas múltiples, formados haciendo el O lógico de NZDA con los bits menos significativos de ADDR en vez de comprobaciones sobre los bits individuales. Además, en muchas máquinas, el traslape de la extracción y la ejecución de las microinstrucciones se hace de tal modo que, al principio de la ejecución de la microinstrucción n , la elección de la microinstrucción $n+1$ ha de hacerse utilizando los bits de estado de la ALU de la microinstrucción $n-1$. No vamos a continuar profundizando en el tema, pero con esto basta para entender completamente la definición de microprogramación dada por Rosin.

4.5.5. Memoria caché

Por tradición, las CPU han sido siempre más rápidas que las memorias y aunque se han desarrollado las segundas, los CPU también lo han hecho, manteniendo así el desequilibrio. Lo que esto significa en la práctica es que después de que la CPU envía una solicitud a la memoria, debe permanecer ociosa durante un tiempo sustancial, aguardando a que la memoria responda. Como se ha visto, es común que la CPU establezca una lectura de memoria durante un ciclo del bus y no obtenga los datos hasta dos o tres ciclos más tarde, aun cuando no haya estados de espera.

En realidad, el problema no es de tecnología sino económico. Los ingenieros saben como construir memorias tan rápidas como las CPU, pero son tan caras que equipar a una computadora con un megabyte o más es impensable (excepto quizá para las supercomputadoras, en las que prácticamente no hay límite y el precio no representa objeción). De ahí que la elección se reduce a tener una pequeña cantidad de memoria o una gran cantidad de memoria lenta. Lo preferible sería una gran cantidad de memoria a un bajo precio.

De sumo interés son las técnicas conocidas para combinar estos tipos de memoria a fin de obtener, a bajo precio, la velocidad de la memoria rápida y la capacidad de la memoria lenta. A la memoria pequeña y rápida se le llama caché (del francés *cacher* que significa esconder) y está bajo el control del microprograma. En seguida, se describirá cómo se utilizan y funcionan estas memorias. Se puede encontrar más información en (Agarwall op. cit., 1989; Farrens y Pleszkun, 1989; Kabakibo op. cit., 1987; Kessler op. cit., 1989; Pohm y Agarwall, 1983; Przybylski op. cit., 1989; Smith, 1982; y Wang op. cit., 1989).

Desde hace años es sabido que los programadores no accesan las memorias en forma completamente aleatoria. Si una referencia a memoria dada, es a la dirección A , es común que la siguiente referencia se realice en los alrededores de

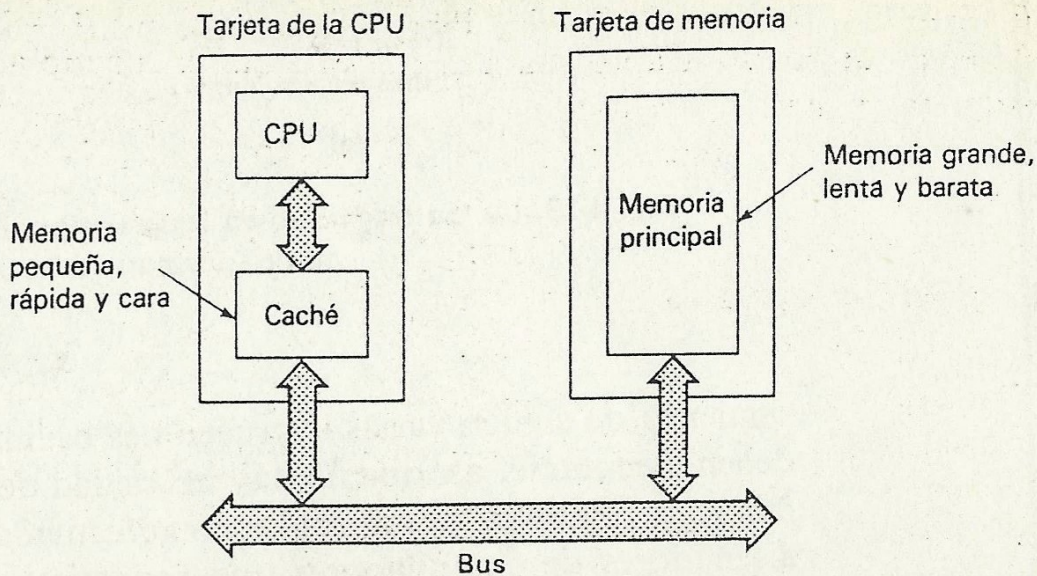


Fig. 4-27. La memoria caché se ubica por lo general en la tarjeta de la CPU.

A. Un ejemplo sencillo es un programa en sí, con excepción de los saltos y las llamadas a procedimientos, las instrucciones se extraen de localidades de memoria consecutivas. Es más, la mayoría del tiempo de ejecución del programa se emplea en iteraciones, en las que un número limitado de instrucciones se ejecutan de manera repetida. En forma parecida, es común que en un programa manejador de matrices se efectúen muchas referencias a la misma matriz, antes de cambiar a otra actividad.

Se le llama **principio de localidad** a la observación de que las referencias a memoria realizadas en un intervalo de tiempo corto, tienden a usar sólo una fracción de la memoria total, y representa la base para todos los sistemas de memoria caché. La idea general es que cuando una palabra es referenciada, se le trae de la memoria grande y lenta hacia la caché, de modo que la siguiente vez que se utilice, se pueda acceder muy rápido. En la figura 4-27 se ilustra una distribución común de la CPU, la memoria principal y la caché. Si una palabra se lee o escribe k veces en un intervalo corto, la computadora necesitará de 1 referencia a la memoria lenta y $k - 1$ referencias a la memoria rápida. Entre más grande es k , mejor es el desempeño general.

Podemos formalizar este cálculo llamado c al tiempo de acceso a la memoria caché, m al tiempo de acceso a la memoria principal y h a la **proporción de aciertos**, que es la fracción de todas las referencias que pueden ser satisfechas fuera de la caché. En el pequeño ejemplo del párrafo previo, $h = (k - 1)/k$, algunos autores definen también la **proporción de fallas** como $1 - h$.

Con estas definiciones podemos calcular el tiempo medio de acceso de la siguiente manera:

$$\text{tiempo medio de acceso} = c + (1 - h)m$$

Cuando h tiende a 1, todas las referencias pueden ser satisfechas en la memoria caché y el tiempo de acceso se aproxima a c . Por otra parte, cuando h tiende a 0, se necesita una referencia a memoria cada vez y el tiempo de acceso se aproxima a $c + m$, primero un tiempo c para verificar (sin éxito) la caché y luego un tiempo m para efectuar la referencia a memoria. En algunos sistemas, la segunda referencia puede iniciarse en paralelo con la búsqueda en la caché, de modo que si no encuentra la palabra en ésta, el ciclo de memoria ya ha comenzado. Sin embargo, esta estrategia es más complicada pues, requiere que la memoria pueda detenerse sobre sus pistas, cuando la palabra fue localizada en la caché. El algoritmo básico de búsqueda en la caché y el iniciar (o detener) la referencia a la memoria principal dependiendo del resultado de dicha búsqueda, son aspectos que maneja el microprograma.

Se utilizan dos formas diferentes de organización de la caché, junto con una tercera que es un híbrido de las dos primeras. Para los tres tipos, se asume que la memoria sea de 2^m bytes, dividida (de manera conceptual) en bloques consecutivos de b bytes, lo que da un total de $2^m/b$ bloques. Cada bloque tiene una dirección que es un múltiplo de b . El tamaño b del bloque es por lo general una potencia de dos.

El primer tipo es la **memoria caché asociativa**, cuyo ejemplo se muestra en la figura 4-28. Consiste en cierto número de **renglones o líneas**, cada uno de los cuales contiene un bloque y su número correspondiente, junto con un bit de *Validez* que indica si el renglón está actualmente en uso o no. El ejemplo de la figura 4-28 ilustra una caché con 1024 renglones y una memoria con 2^{24} bytes, dividido en 2^{22} bloques de 4 bytes. En este tipo de caché el orden de los accesos es aleatorio.

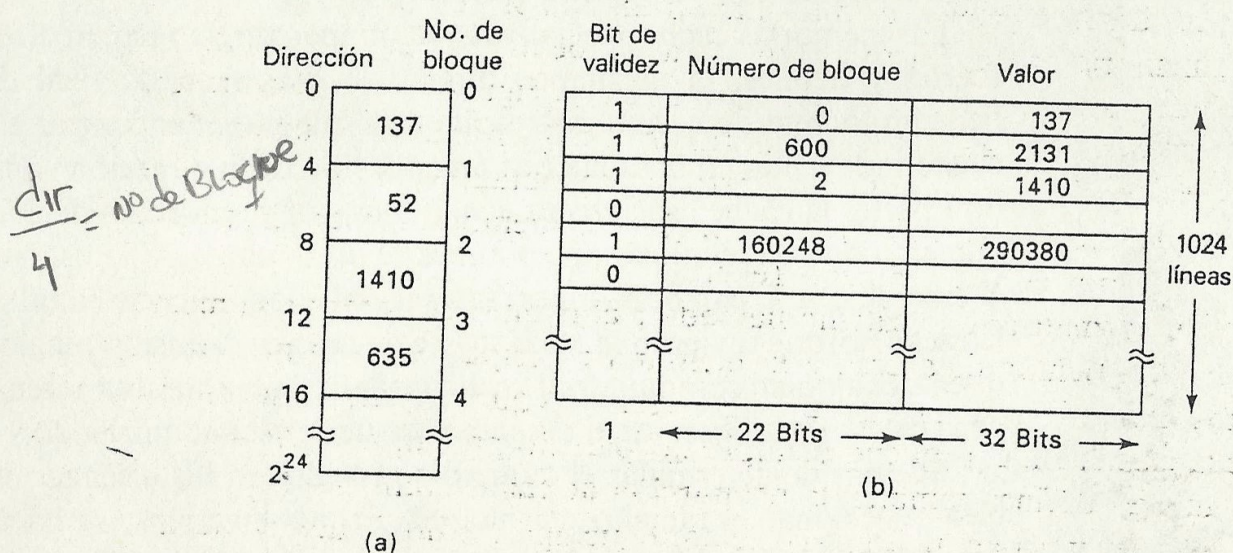


Fig. 4-28. Un ejemplo del esquema de la memoria caché. (a) Memoria con bloques de 4 bytes. (b) Memoria caché asociativa con 1024 líneas.

Al inicializar la computadora, todos los bits de *Validez* toman el valor 0, para indicar que ninguna entrada a la caché es válida. Suponga que la primera instrucción hace referencia a la palabra de 32 bits que se encuentra en la dirección 0. El microprograma checa todos los renglones de la caché buscando uno que sea válido y contenga el bloque número 0. Al no encontrarlo, enviará una solicitud al bus para extraer de la memoria la palabra 0. Si esta palabra se necesita de nuevo, la próxima vez será tomada de la caché, eliminando la necesidad de una operación del bus.

Al correr el tiempo, más y más renglones de la caché se marcarán como válidos. Si el programa completo y sus datos utilizan de 1024 palabras, todo el programa y sus datos aparecerán en la caché y correrá a mayor velocidad, sin hacer ninguna referencia a la memoria a través del bus. Si se necesita más de 1024 palabras, en algún momento la caché estará llena y un renglón previo deberá descartarse para dar lugar a uno nuevo. En la práctica, la decisión de cuál eliminar debe hacerse muy rápido (en nanosegundos). La VAX y muchas otras máquinas seleccionan un renglón en forma aleatoria. En el capítulo 6 se abordan otros algoritmos posibles bajo el título de memoria virtual, en donde ocurre el mismo problema.

El aspecto que distingue a la memoria caché asociativa de los otros tipos, consiste en que cada renglón contiene el número de bloque y su entrada. Cuando se presenta una dirección de memoria, el microprograma debe calcular el número del bloque relevante (fácil) y luego buscar ese número de bloque en la caché (difícil). A fin de evitar una búsqueda lineal, la caché asociativa dispone de un hardware especial que puede comparar cada entrada con el número de bloque dado en forma simultánea, en lugar de hacerlo en un lazo de microprograma. Este tipo de hardware hace que dicha memoria sea cara.

Para reducir el costo, se inventó la **memoria caché de mapeo directo**. Este tipo evita la búsqueda poniendo cada bloque en un renglón cuyo número se puede calcular fácilmente a partir del número del bloque. Por ejemplo, el número de renglón puede ser el número de bloque *módulo* el número de renglones. Con bloques de 4 bytes (una palabra) y 1024 renglones, el número de renglón que corresponde a la dirección A es igual a $(A/4) \text{ módulo } 1024$. En el ejemplo de la figura 4-29 se puede observar que las palabras en las direcciones 0, 4096, 8192, etc., caen en el renglón 0; las palabras en las direcciones 4, 4100, 8196, etc., se mapean al renglón 1 y así sucesivamente.

Mientras que las memorias caché de mapeo directo eliminan el problema de buscar, crean por otro lado un nuevo problema; cómo indicar cuál de las muchas palabras que corresponden al renglón lo está ocupando. En efecto, se han creado 1024 clases de equivalencia basada en los números de bloque *módulo* el tamaño de la caché. En el ejemplo, el renglón 0 puede contener cualquiera de las palabras 0, 4096, 8192, etc., la forma de señalar cuál está en un momento dado en dicho renglón es poner parte de la dirección en el campo *etiqueta* de la caché.

HASHIN
 $\frac{dir}{4} \% 1024$
 mob

Renglón	Bit de validez	Etiqueta	Valor	Direcciones que usan este renglón
0	1	2	12130	0, 4096, 8192, 12288, ...
1	1	1	170	4, 4100, 8196, 12292, ...
2	1	3	2142	8, 4104, 8200, 12296, ...
3	0			12, 4108, 8204, 12300, ...
4	0			16, 4112, 8208, 12304, ...
5	0			20, 4116, 8212, 12308, ...
...	...	...	...	...
1023	1			4092, 8188, 12284, ...

(a)

Bits	12	10	2
	Etiqueta	Renglón	00

(b)

Fig. 4-29. (a) Una memoria caché de localización directa, con 1024 renglones de 4 bytes cada uno. (b) Cálculo del renglón y etiqueta de una dirección de 24 bits.

Este campo contiene aquella parte de la dirección que no puede calcularse a partir del número de renglón.

Para hacer este punto más claro, considérese una instrucción en la dirección 8192 que mueve la palabra ubicada en la dirección 4100 a la 12296. El número de bloque correspondiente a la dirección 8192 se calcula dividiéndola entre 4 (tamaño del bloque del ejemplo), para obtener 2048. En seguida, el número de renglón se calcula como $2048 \bmod 1024$, lo que es lo mismo que usar 10 bits de orden inferior de 2048. El número de renglón es 0. Los 12 bits de orden superior contienen un 2, de modo que esa es la etiqueta. La figura 4-29(a) muestra a la caché después de haber procesado las tres direcciones.

En la figura 4-29(b) se puede ver la dirección en forma separada. Los dos bits de orden inferior son siempre 0 (ya que la caché trabaja con bloques enteros y éstos son múltiplos del tamaño del bloque, 4 bytes en el ejemplo). En seguida está el número del renglón (10 bits) y por último la etiqueta (12 bits). Es muy fácil construir hardware que extraiga directamente el número de renglón y la etiqueta, de cualquier dirección de memoria.

El hecho de que muchos bloques se mapeen en un mismo renglón puede causar problemas. Supóngase que la instrucción moviera la dirección 4100 a la 12292 en lugar de a la 12296; ambas direcciones se mapearían al renglón 1. Dependiendo de los detalles del microprograma, la dirección que se calculara después permanecería en la caché, mientras que la otra sería purgada. Esto no es un desastre en sí pero sí se degrada el desempeño de la caché si muchas palabras

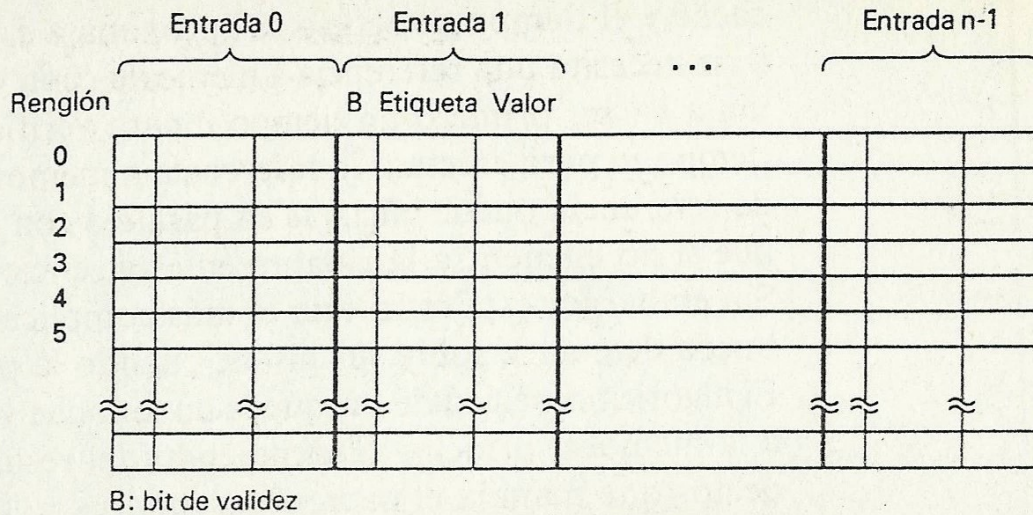


Fig. 4-30. Una memoria caché asociativa agrupada con n entradas por renglón.

de las utilizadas resulta que están en el mismo renglón. La meta, después de todo, es mejorar el rendimiento.

La salida de esta dificultad consiste en expandir la caché de mapeo directo con más de un registro por renglón. La PDP-11/70, por ejemplo, tenía dos registros por renglón. A la memoria caché con varios registros por renglón se le denomina **memoria caché asociativa agrupada** y se ilustra en la figura 4-30.

Tanto la caché asociativa como la de mapeo directo son de hecho, casos especiales de la asociativa agrupada. Si se reduce el número de renglones a 1, todos los registros de la caché estarán en el mismo renglón y habrá que distinguirlos sólo por sus etiquetas ya que todos caen en la misma dirección. Este caso es sólo una caché asociativa. Si $n = 1$ se regresa a una memoria caché de mapeo directo con un registro por renglón.

Las memorias caché asociativas y de mapeo directo tienen diferentes puntos fuertes y débiles, la de mapeo directo es más sencilla, fácil de construir y tiene un tiempo de acceso más rápido, ya que puede encontrar el renglón adecuado indexando en la caché con una porción de la dirección como índice. Por otra parte, la caché asociativa tiene una mayor proporción de aciertos para un número dado de renglones, pues nunca se presentan conflictos. No se puede dar el caso de que k palabras importantes no puedan ubicarse simultáneamente en la caché porque tuvieron la mala suerte de caer en el mismo renglón. Cuando se diseña cualquier computadora real, se realiza siempre una intensa simulación de la caché para observar el desempeño que se está adquiriendo y a que precio.

Además de determinar el número de renglones, los diseñadores deben también seleccionar el tamaño del bloque. En los ejemplos, se ha utilizado (por simplicidad) una palabra de 32 bits, pero también es posible y se usan con frecuencia 2, 4, 8, o más palabras. Una ventaja de usar un tamaño grande de bloques es que hay menor gasto general para extraer un bloque de ocho palabras que para ocho bloques de una palabra, en especial si el bus permite transferen-

cias en bloque. La desventaja es que puede no necesitarse todas las palabras, así que algunas extracciones pueden desperdiciarse.

Otra parte importante en el diseño de la caché es cómo manejar las escrituras. Por lo general se utilizan dos estrategias. En la primera denominada **con escritura a memoria**, cuando una palabra se escribe en la caché de inmediato se escribe en la memoria. Este enfoque asegura que los registros en la caché son siempre los mismos que los correspondientes a la memoria.

La otra política de escritura llamada **retrocopiado**, no actualiza la memoria siempre que cambia la caché, sino que sólo lo hace cuando un registro es eliminado de la caché para dar lugar a que uno nuevo ocupe ese renglón. Cuando se utiliza el retrocopiado, es necesario un bit en cada registro de la memoria caché que indique si éste ha cambiado desde que se cargó en ésta.

Al igual que en todos los demás aspectos de diseño de la caché, existen aquí ciertas consideraciones. Es obvio que con escritura a memoria cause más tráfico del bus que el retrocopiado. Por otra parte, si una CPU inicia una transferencia de entrada/salida de memoria a disco y la memoria está incorrecta (porque palabras que se modificaron recientemente aún no han sido copiadas a memoria desde la caché), se escribirán datos incorrectos en el disco. Este problema se puede evitar, pero agrega complejidad al sistema.

Si el porcentaje de lecturas es más alto que el de escrituras, puede resultar más simple usar la caché con escritura a memoria y aceptar el tráfico del bus. Sin embargo, si existen muchas escrituras, pudiera ser mejor utilizar el retrocopiado y hacer que el microprograma explícitamente limpie la caché por completo antes de iniciar una operación de E/S.

Otro aspecto de diseño relacionado con las escrituras, es qué hacer si una escritura provoca una pérdida en la caché. Una estrategia consiste en extraer la palabra en la caché y luego actualizarla, si se está usando caché con escritura a memoria. Una estrategia diferente es escribir directamente a memoria, pero no extraer la palabra a la caché excepto en lecturas. A este aspecto se le conoce con el nombre de **política de asignación de escrituras**. Ambos métodos descritos se encuentran en uso.

Al mejorar la tecnología de los circuitos integrados, ha sido posible poner cachés muy rápidas, directamente en la pastilla de la CPU. Debido a la falta de área en la pastilla, estas memorias son pequeñas, así que sería desable tener dos niveles de caché, una en la pastilla de la CPU y otra en la tarjeta. Si no se puede encontrar una palabra en la caché integrada en la pastilla, se hace una solicitud a la caché en el nivel de tarjeta. Si falla, se utiliza la memoria principal.

Considerando todo lo anterior, el uso de la memoria caché es una técnica importante para incrementar el rendimiento de un sistema. Casi todas las computadoras de mediana y gran escala utilizan alguna forma de esta técnica. Se puede encontrar información adicional sobre el tema en (Hill, 1988; Przybylski op. cit., 1988; Short y Levy, 1988; y Smith, 1986, 1987).